

Lecture #1

Data types

- First 3 weeks $\Rightarrow$ revise on JS in general, you'll find that you know most/ of the information but you'll find some ^{some} tricks on your way will help you understand behind the scenes all ~~what's~~ why you are doing that.

Goal of the course ① Once you read the JS code, you know its purpose & understand what it does

② Know the steps the engine passes by

③ Why I write that code & not that

④ Performance based on DSA & Calculate Big O

① Data Types

JS $\Rightarrow$ 1995

type null $\Rightarrow$ object

BigInt

Symbol

ECMAScript

TC 39 $\Rightarrow$ Committee responsible for developing and evolving the ECMAScript Specification

TC = Technical Committee 39, No. inside org.

You should follow the community news *

→ Don't count on some one or a tool ^{Ever} _{Never!}

→ Structured Clone is better than `JSON.Parse()`

bec. JSON serialization
has many limitations and can lose data

`JSON.Parse(JSON.stringify(obj))`

```
const user = {
```

```
  name: "Omar",
```

```
  age: 28,
```

```
  createdAt: new Date(),
```

```
};
```

```
const cloned = JSON.parse(JSON.stringify(user));
```

```
console.log(cloned.createdAt) // string, NOT DATE anymore
```

It loses `Map`, `Set`, circula reference throws error.

→ Structured Clone still can't clone functions
bec. functions are executable behaviour, not pure data

→ Reach to a Mindset of a SWE!,
Don't depend on a course to implement
sth.

! ثقافة المبرمج

→ know a thing on every thing &
know every thing on a thing!

TC39
↓
important

→ Watch Crash Courses!



Native: This means the language itself - the core features defined by the ES specification.

Math.random, String, Number, Boolean, Function, variables, fns, obj, arr, promises, classes, async, await.

↳ Can be used anywhere (FE, BE, AI, Automation)

TC39 defines these

Hosted & Hosted Features are APIs provided by the ~~env~~ environment hosting JS.

→ JS alone does not contain document, window, fetch, set Timeout.

These ~~are~~ come from the host environment.

Browser - Hosted JS, ⇒ The browser provides APIs such as:

document, query selector(), window.alert(),
local Storage, fetch()

These are called Web APIs.

The browser injects them into the JS env.

Node JS env.

env give JS Power!

But fn is not a group of primitives"

// data types

It's an executable object

Var x = "ay faga"
x = 10 } loosely-typed &
Dynamic language

// primitive data type & Non-Primitive

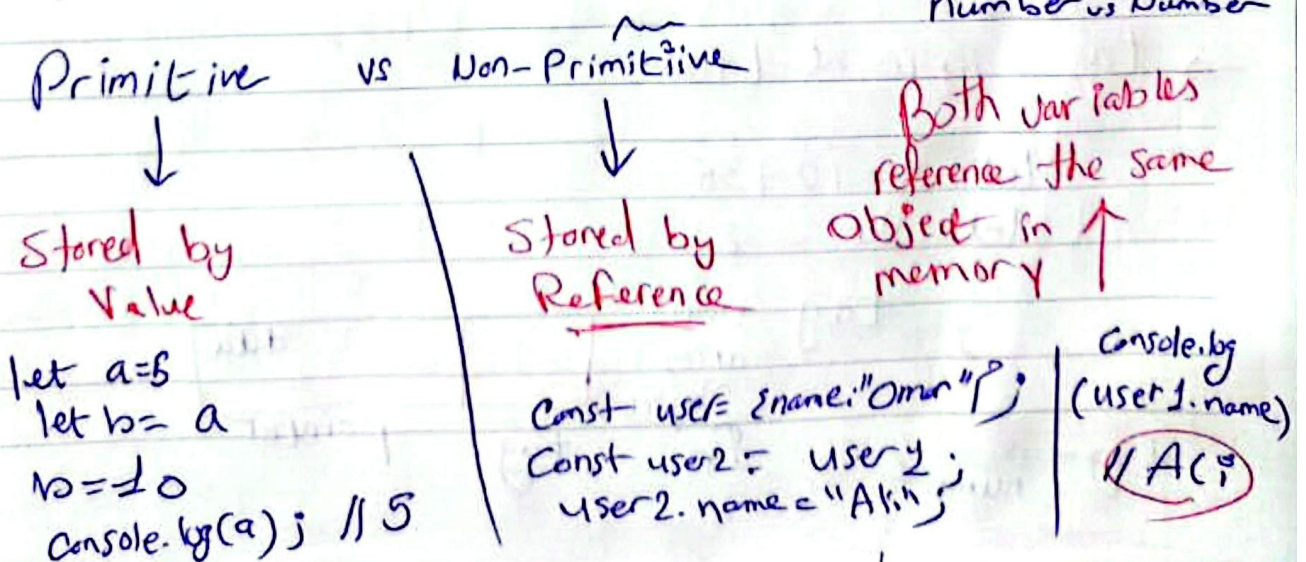
→ Some books say that non-primitive data types are groups of primitive ^{Composite}

→ Primitive values are simple standalone values
→ Non-Primitive are complex/reference-based structure that can store and organize values.

Primitive: string, number, boolean, null, undefined
atomic Symbol, bigint

Non-Primitive: object, array, function, date, map, set

→ In Js, primitive types can be "wrapped" into non-primitive objects (int vs Integer)
Number vs Number



gives error

octal

↓

Base 8

let $x = 0.123$;

you should add 'o'

→ 0 1 2 3 4 5 6 7
← 8^2 8^1 8^0
1 2 3 1

1 x	$\rightarrow 8^2$	64	
2 x	$\rightarrow 8^1$		16
3 x	$\rightarrow 8^0$		3

Hexa = 0x

binary = 0b

decimal $\rightarrow$ none $123_{10} = 1 \times 10^2 + 2 \times 10^1 + 3 \times 10^0$

Why "null" is object?

let $x = \text{null}$;

`console.log(typeof x);` // object → why ??!

→ tag representation

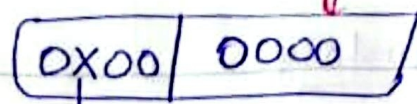
let $x = 12736$

let user = 2

3 tag 000
array
object
fn

tag pointer

tag $\Rightarrow$ number



000	0000
-----	------

→ This is a problem in JS since built.

Why not solved till now? bec it will corrupt many applications

more in the article medium

Party: null vs undefined

undefined $\rightarrow$ A value was not assigned yet
null $\rightarrow$ An empty value was intentionally assigned

No need cliff $\rightarrow$ it's based on arrangement with the Backend ✓ Contract

console.log (null === undefined) False
(==) True

JS also automatically gives undefined in many situations.

```
let name; console.log(name); // undefined.  
const user = {?, user.age,  
function test(){? console.log(test());
```


(undefined)

Simple analogy

Imagine filling a form: No one touched the field yet
null $\rightarrow$ Someone intentionally cleared the field

Console.log(Number.MAX_SAFE_INTEGER)

The value of the largest integer n such that n and $n+1$ are both exactly representable as a Number value.

Its value is $2^{53} - 1$

Solution to hold bigger numbers

BigInt $\rightarrow$ 9007199254740993n

up is also a

BigInt

let x = 10n

let y = 5

Console.log(x+y) ~~is~~ Error $\rightarrow$ TypeError

Cannot mix BigInt and other types X

Solution: Use Explicit Object conversions

Console.log(x + BigInt(y));

or the opposite, but you may lose precision again! take care!!!

Summary: Number in JS has limitation

if you passed it, will return you back to that limitation

BigInt $\rightarrow$ used in AI and Machine Learning a lot.

~~~~~ bec. you train your model on huge data

so of course it may exceed the SAFE-INTEGER.



Symbol  $\Rightarrow$  unique identifier

// UUID

```
let x = "nowr";
```

```
let y = "nowr";
```

```
console.log (x == y); // true
```

Solution: if I don't want them to be equal (unique)

```
let x = Symbol ("nowr");  $\rightarrow$  returns unique id
```

```
let y = Symbol ("nowr");  $\rightarrow$  " " "
```

```
console.log (x == y); // false
```

Pop- Quiz!

```
let user = {  
  name: "Ahmed",  
  age: 33,  
  name: "Abdo"  
}
```

~~~~~ what will happen here ???

It will take the latest

Abdo

→ Solution if I want to create a unique Property
so no one can override it

```
let key = Symbol("id");
```

```
let user = {  
  name: "Ahmed",  
  age: 33,  
  [key]: "Abdo"  
};  
⇒ {  
  name: "Ahmed",  
  age: 33,  
  Symbol(id): "Abdo"  
}
```

Object.keys ⇒ name, age only

That's good bec. no one can override that unique key except you, only if you know it exactly, also it has another way of accessing it.

How to access it? Object.getOwnPropertySymbols
↓
[Symbol(id)] user

user[key] ⇒ Abdo

uuid versions has to be arranged with the BE

Type Conversions

حرفاء في JS

↓ Coercion

Implicit, explicit

فرضي
الجدول نفسي
لو جدي

I convert it myself

Explicit

```
let x = "123";
console.log(Number(x)); // 123
// " (String) (Number(123))
```

I used methods and constructor functions to convert it manually

Concat

Implicit

console.log("5" + 1);

js ==> String & 8 + ==> Convert other values to string

console.log("5" - 1); // js ==> String 8 - ==> Num
Tries to convert it to a number

or
- 1 * --

special case

"hour" * 2 ==> NaN -> not a number type ==> number

Interview tricks

value of 0.

↳ toString $\Rightarrow$ type string

$[] + [] \Rightarrow ""$

$(["Omar", 2], toString)$

$\Rightarrow "Omar, 2"$

$[] + \{ \}$ " [object Object]"

$true + true$ 2

$"5" + true$ "5true"

in case of
arrays, objects

$null + 1$ 1

$undefined + 1$ NaN

Next lecture Insha'Allah

1- ~~Loops~~ Loops with respect to Performance
for in, for of, for $\rightarrow$ normal

2- Pre-allocated array, Push(), Type Array

3- Generators

4- break, continue label, hole array

(large scale array)

let x = [] vs new Array(4)

(let, const, var, scope)

Performance